1. BASICS OF JAVA.

➢ What is Java?

- Java is an **object-oriented programming language**.
- In Java, **everything revolves around classes and objects**.

➢ Simple hello world program in java

```
class HelloWorld

{

    public static void main(String[] args)

    {

      System.out.println("Hello World");

   }

}
```

Output:

Hello World

➢ Explanation:

class HelloWorld → creates a class (same name as file)

public static void main(String[] args) → program starts from here

System.out.println() → prints output on the screen

File name: HelloWorld.java

➢ Why main method is static?

- **Main method is static so that JVM can call it without creating an object.**
- **"Main method is static because it is called by JVM without object creation."**

➢ Can Java program run without class?

Short Answer:

✕No

Explanation:

- Java is **object-oriented**
- Every Java program must be written **inside a class**
- JVM starts execution from **main method inside a class**

Example:

System.out.println("Hello"); // ✕Invalid

```
class Test
{
   public static void main(String[] args)
   {
      System.out.println("Hello"); // ✓Valid
   }
}
```

Important note :

- *Even if we don't write class, compiler will not allow program.*
- **"Java program cannot run without class."**

➢ Who calls the main method?

- **JVM (Java Virtual Machine) calls the main method.**

1.1 Class and Objects

- Java follows the **Object-Oriented Programming (OOP)** approach.
- OOP is based on the idea of:

  Creating **classes**

  Using those classes to create **objects**

  Solving real-world problems by representing them as objects

- **Real-World Example:**

  Class: Car

  Objects: BMW, Audi, Swift

  Properties: color, speed, model

  Behaviors: start(), stop(), accelerate()

➢ What is a Class?

- A **class** is a **user-defined data type** that acts as a **blueprint** for creating objects.
- So, class is a collection of data members and member functions.
- It defines:

  Data members (variables / fields)

  Member functions (methods)

➢ Components of a Class

1. Variables (Data Members)

- Variables store data related to the object.
- Example:

```
int id;
String name;
```

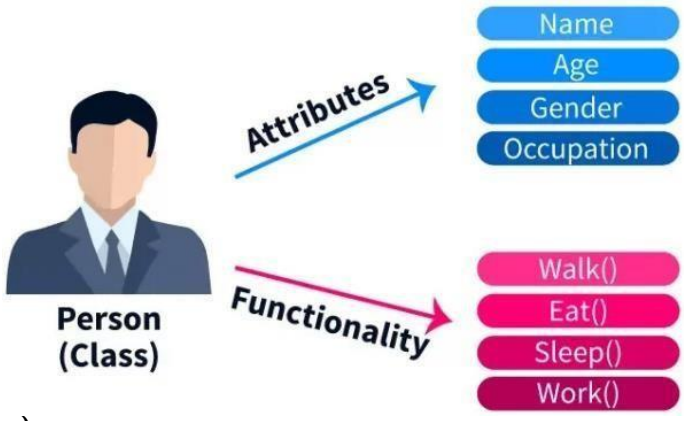
2. Methods (Member Functions)

- Methods define the behavior of an object.
- Example:

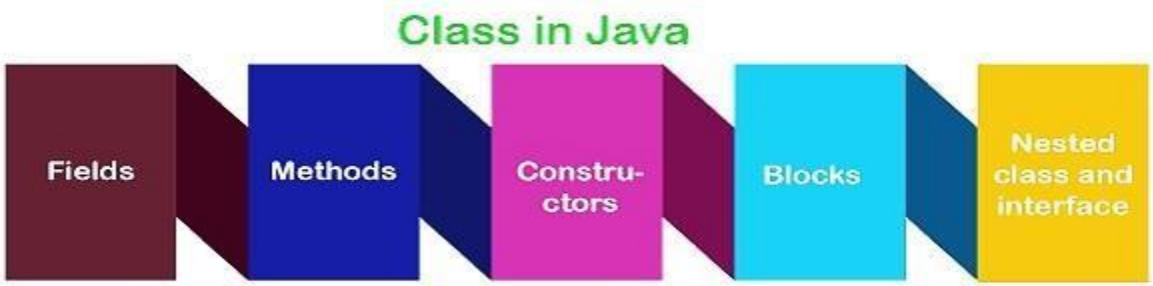
Example of Class

```
class Student
{
   int rollNo;
   String name;

    void show()
    {
      System.out.println(rollNo + " " + name);
     }
}
```



- A Class in Java can contain:



1. Member/ Fields/ Instance Variables

- These are the properties or attributes of the class, which represent the state of objects created from the class.
- Example:

```
int age;
String
name;
```

2. Method

- These define the behavior of the objects.
- Methods are functions inside a class that can operate on the class's fields.

```
void displayDetails()
 {
System.out.println("Name: " + name + ", Age: " + age);
 }
```

3. Constructor

- Special methods used to initialize objects.
- A constructor has the same name as the class and no return type.
- Example:

```
class Student
{
   int rollNo;
   String name;

   Student(int r, String n)
   {
      rollNo = r;
```

```
        name = n;
    }
    public static void main(String[] args)
    {
        Student s = new Student(101, "Amit");
        System.out.println(s.rollNo + " " + s.name);
    }
}
```

4. Access Modifiers

- Define the visibility of the class, its methods, and fields.
- Examples include public, private, protected, and private (default).

5. Interface

- An **interface** in Java is a **blueprint of a class** that is used to achieve **100% abstraction**.
- It contains:

  Abstract methods (methods without body)

6. Nested Class

- A **nested class** in Java is a **class defined inside another class**

➢ What is an Object?

- An **object** is a **real-world instance** of a class.
- It represents **actual data**
- It occupies **memory**
- Multiple objects can be created from the same class
- **Syntax of Object Creation:**

ClassName objectName = new ClassName();

Where 1) ClassName:

This is the **name of the class**

It tells Java **which class object we want to create**

Example:

Student

2)objectName:

This is the **reference variable**

It stores the **address of the object**

Used to access data members and methods

Example:

Student s1;

3) = Assignment operator

Assigns the object's reference to objectName

4) **new: new keyword creates memory in heap**

It creates a **new object**

5)ClassName()

This is the **constructor call**

Constructor initializes the object

Called **automatically**

- Example:

Student s1 = new Student();

Accessing Class Members Using Object

The **dot (.) operator** is used.

```
s1.rollNo = 101;
s1.show();
```

Example 1: Write a java program by using class to display student information.

```
class Student
{
   int rollNo;
   String name;

    void display()
    {
      System.out.println("Roll No: " + rollNo);
      System.out.println("Name: " + name);
   }

    public static void main(String[] args)
    {
      Student s1 = new Student();

      s1.rollNo = 101;
```

```
        s1.name = "Rahul";

        s1.display();
    }
}
```

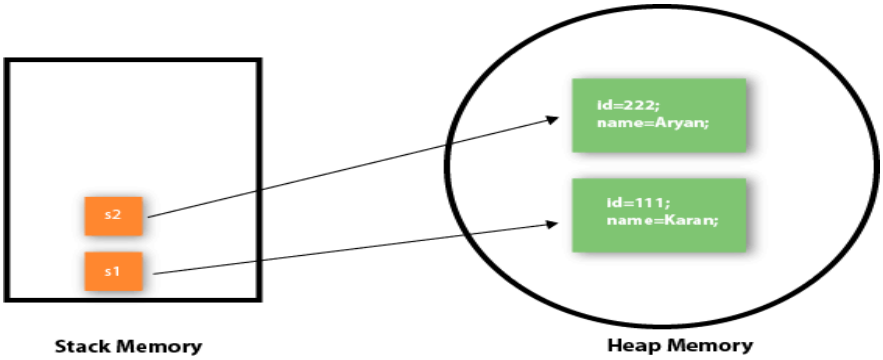
Explanation:

Student → class

s1 → object

Variables store student details

Method displays the data



What is Heap Memory?

- Heap is a **runtime memory** area
- Used to store **objects and instance variables**
- Memory is allocated using the new **keyword**
- It is **shared** among all threads

➢ Multiple Objects of Same Class

- One class can have **multiple objects**.
- Each object:

  Has **separate memory**

  Can store **different values**

Example 2: Multiple Objects

```
class Employee
{
   int empId;
   String empName;

   void show()
   {
      System.out.println(empId + " " + empName);
   }

   public static void main(String[] args)
   {
      Employee e1 = new Employee();
      Employee e2 = new Employee();

      e1.empId = 1;
      e1.empName = "Amit";

      e2.empId = 2;
      e2.empName = "Neha";

      e1.show();
      e2.show();
   }
}
```

Example 3: Write a java program for addition of 2 numbers

```
class addition
{

     void add()
     {
        int a = 5;
        int b = 10;
       int c=a+b;
        System.out.println("Sum = " + c);
    }

     public static void main(String[] args)
     {
        addition a=new addition();
        a.add();
    }
}
```

Example 3: Write a java program for addition of 2 numbers. (Take input from user)

```
import java.util.Scanner;

class Addition
{
   public static void main(String[] args)
   {
      int num1, num2, sum;

      // Create Scanner object
      Scanner sc = new Scanner(System.in);

      // Take input from user
      System.out.print("Enter first number: ");
      num1 = sc.nextInt();

      System.out.print("Enter second number: ");
      num2 = sc.nextInt();

      // Perform addition
      sum = num1 + num2;

      // Display result
      System.out.println("Sum of two numbers = " + sum);
   }
}
```

Output:

```
Enter first number: 10
Enter second number: 20
Sum of two numbers = 30
```

Explanation:

Java Code line by line explanation

1. import java.util.Scanner;

- Imports the Scanner class from the java.util package.
- Scanner is used to take input from the user through the keyboard.

2) class Addition

- Defines a class named **Addition**.
- In Java, every program must have at least one class.

3) {

- Opening brace of the class.

4) public static void main(String[] args)

- This is the **main method**.
- Program execution starts from here.
- public → accessible everywhere
- static → no object needed to run
- void → does not return any value
- String[] args → used for command-line arguments

5) {

- Opening brace of the main method.

6) int num1, num2, sum;

- Declares three integer variables:
- num1 → first number
- num2 → second number
- sum → stores the result of addition

7)Scanner sc = new Scanner(System.in);

- Creates a **Scanner object** named sc.
- System.in means input will be taken from the keyboard.

8)System.out.print("Enter first number: ");

- Displays the message asking the user to enter the first number.

9) num1 = sc.nextInt();

- Reads an integer value entered by the user.
- Stores it in the variable num1.

10) System.out.print("Enter second number: ");

- Displays the message asking the user to enter the second number.

11)num2 = sc.nextInt();

- Reads the second integer value from the user.
- Stores it in the variable num2.

12)sum = num1 + num2;

- Adds num1 and num2.
- Stores the result in the variable sum.

13)System.out.println("Sum of two numbers = " + sum);

- Prints the final result on the screen.
- + is used to concatenate text with the variable value.

14) }

- Closing brace of the main method.

15) }

- Closing brace of the class.

➢ DIFFERENCE BETWEEN JAVA CLASSES AND OBJECTS:

Class	Object
Class is the blueprint of an object. It is used to create objects.	An object is an instance of the class.
No memory is allocated when a class is declared.	Memory is allocated as soon as an object is created.
A class is a group of similar objects.	An object is a real-world entity such as a book, car, etc.
Class is a logical entity.	An object is a physical entity.
A class can only be declared once.	Objects can be created many times as per Requirement.
An example of class can be a car.	Objects of the class car can be BMW, Mercedes, Ferrari, etc.

Example:
class Student
{
int rollNo;
String name;
}
	Example:
	Student s = new Student();

1.2Abstraction in Java

1.2.1 What is Abstraction?

- **Abstraction** means **hiding implementation details** and **showing only essential features** to the user.
- Abstraction means **Hiding internal implementation and showing only required functionality.**
- User knows **what a method does**
- User does **not know how it is implemented**

Definition :

Abstraction is the process of hiding implementation details and exposing only essential features.

What to do is shown, *how to do* is hidden.

Real-Life Example :

Example 1: When you drive a car:

You use **steering, brake, accelerator**

You don't know **internal engine working**

This is **abstraction**

Example 2 : ATM Machine

You see: **withdraw, deposit, balance**

You don't see: **bank server logic**

Example 3: Mobile Phone

You press **call**

You don't know **network routing**

This is abstraction.

➢ Why Abstraction is Needed?

Reduces complexity

Improves security

Makes code easy to understand

Helps in code maintenance

➢ How to Achieve Abstraction in Java?

Java provides **two mechanisms**:

Mechanism	Abstraction Level
Abstract Class	Partial abstraction
Interface	Full abstraction

Real-world examples:

1. The first example, let's consider a Car, which abstracts internal details and exposes to the driver only those de that are relevant to the interaction of the driver with the C

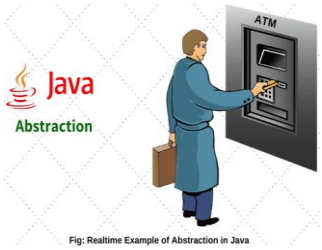
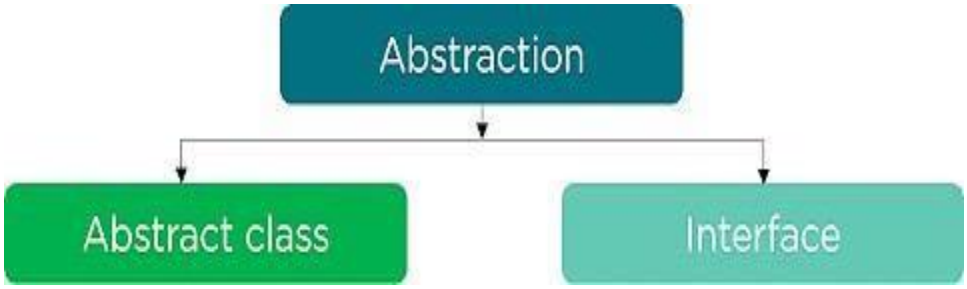


Fig: Realtime Example of Abstraction in Java

2. The second example, consider an ATM Machine; All are performing operations on the ATM machine like cash withdrawal, money transfer, retrieve mini-statement...etc. but we can't know internal details about ATM.



1.2.2 Polymorphism in Java

➢ Meaning of Polymorphism

- **Word Meaning**

  Poly → many

  Morph → forms

- **Polymorphism = one thing behaving in many forms**
- In Java, polymorphism refers to the ability of a message to be displayed in more than one form.

Definition :

Polymorphism in Java is an OOP concept where **one method, object, or reference variable can perform different actions depending on the object it refers to**.

Example 1: Real-life Illustration of Polymorphism in Java:

A person can have different characteristics at the same time. Like a man at the same time is a father, a husband, and an employee. So the same person possesses different behaviors in different situations. This is called polymorphism.

Example 2:

- The + operator is used to **add values and join strings**.
- **Arithmetic Example Sentence**

  The + operator adds two numbers, for example 10 + 20 = 30.

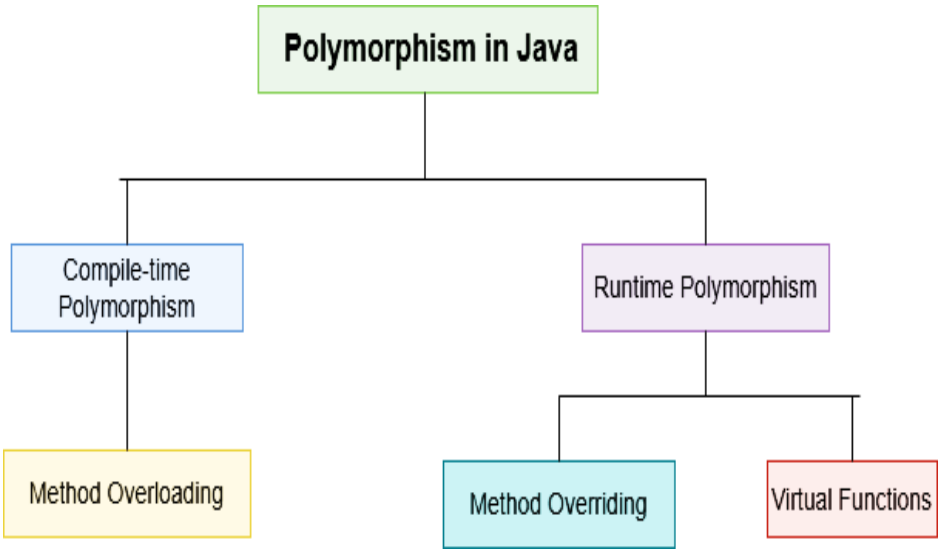
- **String Concatenation Sentence**

  In Java, the + operator is used to concatenate strings, such as "Hello" + " World" which results in "Hello World".

➢ Types of Java Polymorphism:

In Java Polymorphism is mainly divided into two types:

1. **Compile-Time Polymorphism**
2. **Runtime Polymorphism**



1. Compile-Time Polymorphism (Method Overloading)

- **Definition**

  When **multiple methods have the same name** but **different parameters** in the same class, it is called **method overloading**.

- **Compile-Time Polymorphism** is a type of polymorphism where the **method call is resolved at compile time**.
- It is achieved using **method overloading** in Java.
- Also called **Static Polymorphism**.

How It Is Achieved

- Compile-time polymorphism is achieved by:

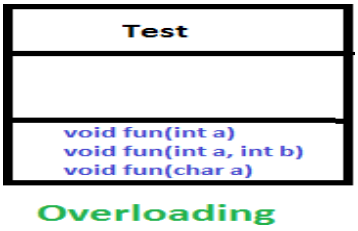
  Method Overloading

  Operator Overloading (limited in Java, e.g., + operator)

Method Overloading

- **Method Overloading** occurs when a class has many methods with the **same name** but **different parameters**.
- Two or more methods may have the same name if they have other numbers of parameters, different data types, or different numbers of parameters and different data types.
- Example

```
void fun() { ... }// No Parameter List
void fun(int num1 ) { ... }//One Parameter
void fun(float num1) { ... }//One Parameter but different datatype
void fun(int num1 , float num2 ) { ... }        //Two Parameter
```



Example 1: Method Overloading

a) **Define a class calculation to implement method overloading for addition of two integers and two double variables. [5]**

[Previous year paper Question: May 2025]

```
public class Calculation
{

   // Method to add two integers
   void add(int a, int b)
   {
      int c;
      c=a+b;
      System.out.println("Addition of integers: " +c);
   }

   // Method to add two double values
   void add(double a, double b)
   {
      double sum;
      sum=a+b;
      System.out.println("Addition of doubles: " +sum);
   }

   public static void main(String[] args)
   {

      Calculation obj = new Calculation();

      obj.add(10,20);       // calls int method
      obj.add(10.23,23.456);  // calls double method
   }
}
```

Output:

Addition of integers: 30

Addition of doubles: 33.686

Rules of Method Overloading

1. Method name must be same
2. Parameter list must be different
3. Can have different return types
4. Can be overloaded in same class or subclass

2. Runtime Polymorphism: (Method Overriding)

➢ **What is Runtime Polymorphism?**

- **Runtime polymorphism** is a type of polymorphism where:
- Method call is **decided at runtime**
- It uses **method overriding**
- JVM decides **which method to execute** while the program is running
- It is also called **Dynamic Method Dispatch**.

➢ What is Method Overriding?

- **Method overriding** means:
- A **child class redefines a method** of its parent class
- Method name, parameters, and return type remain **same**
- Method implementation (logic) is **different**

- **Definition**

When **method name is same**, **parameters are same**, but **implementation (body) is different**, it is called **method overriding**.

➢ Basic Conditions for Runtime Polymorphism

To achieve runtime polymorphism, **three things are compulsory**:

1. Inheritance

One class must extend another class.

2. Method Overriding

Child class must override the parent class method.

3. Parent Class Reference

Parent class reference(object) must point to child class object.

Example: Method Overriding/Run Time polymorphism

```
//Parent class
class Animal
{
   void sound()
   {
      System.out.println("Animal makes a sound");
   }
}
//Child class
class Dog extends Animal
{

   void sound()
   {
      System.out.println("Dog barks");
   }
}

public class test
{
   public static void main(String[] args)
   {
      Animal a = new Dog(); // Runtime polymorphism
      a.sound();
   }
}
```

Output

Dog barks

Explanation:

Why Parent Reference is Used?

Animal a = new Dog();

This enables: Runtime polymorphism

Flexible Code Example

```
Animal a;

a = new Dog();
a.sound();

a = new Cat();
a.sound();
```

Same reference, different behavior
No code change needed

➢ Method Overloading vs Method Overriding:

Basis	Method Overloading	Method Overriding
Definition	Same method name with **different parameter list**	Same method name and **same parameters**, but **different implementation**
Polymorphism type	**Compile-time polymorphism**	**Runtime polymorphism**
Time of binding	Method call resolved at **compile time**	Method call resolved at **runtime**
Class involvement	Occurs in the **same class**	Occurs in **parent–child classes**
Inheritance	**Not required**	**Mandatory**
Method signature	Must be **different** (parameters differ)	Must be **same**
Parameters	Number, type, or order must differ	Must be exactly same
Return type	Can be different (if parameters differ)	Same
Performance	Faster (early binding)	Slightly slower (late binding)
JVM involvement	No runtime decision	JVM decides method at runtime

➢ Inheritance:

- Inheritance is an important pillar of OOP(Object-Oriented Programming).
- Inheritance is a mechanism in Java where
  one class acquires the properties and behaviors (variables and methods) of another class.
- It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another class.
- In Java, Inheritance means creating new classes based on existing ones.
- A class that inherits from another class can reuse the methods and fields of that class.
- In addition, you can add new fields and methods to your current class as well.

Why Do We Need Java Inheritance?

1. Code Reusability:

- The code written in the Superclass is common to all subclasses.
- Child classes can directly use the parent class code.

2. Method Overriding:

- Method Overriding is achievable only through Inheritance.
- It is one of the ways by which Java achieves Run Time Polymorphism.

3. Abstraction:

- The concept of abstract where we do not have to provide all details, is achieved through inheritance.
- Abstraction only shows the functionality to the user.

➢ How to Use Inheritance in Java?

- The **extends** keyword is used for inheritance in Java
- Using the **extends** keyword indicates you are derived from an existing class. In other words, "**extends**" refers to increased functionality.

Syntax:

```
class DerivedClass extends BaseClass
{
  //methods and fields
}
```

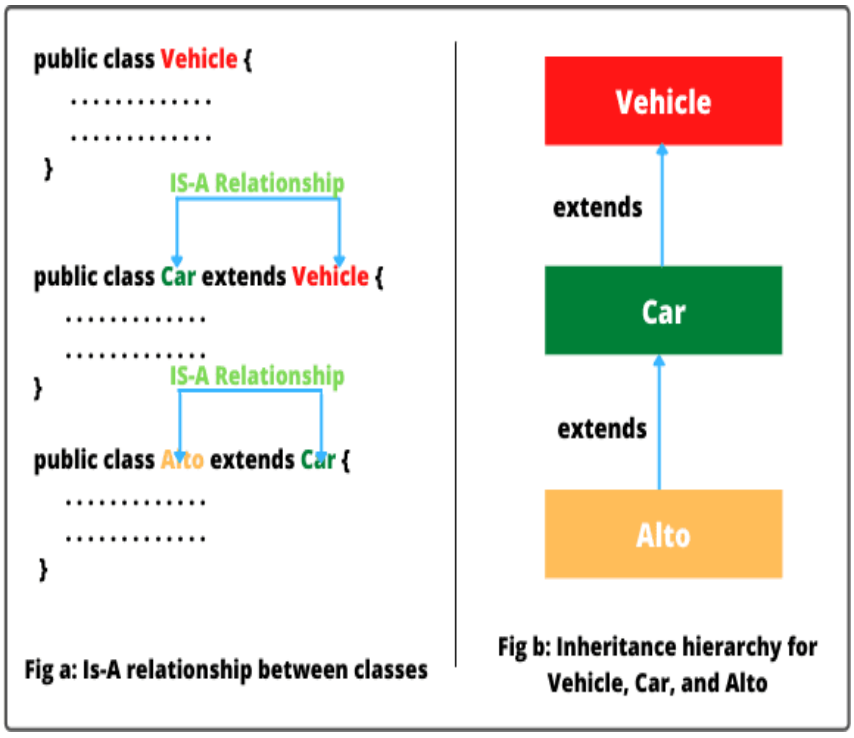


Fig a: Is-A relationship between classes

Fig b: Inheritance hierarchy for Vehicle, Car, and Alto

Example:

```
class Animal
{
   void eat()
   {
      System.out.println("Animal eats food");
   }
}

class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog barks");
   }
}

public class InheritanceExample
{
   public static void main(String[] args)
   {
      Dog d = new Dog();
      d.eat();
      d.bark();
   }
}
```

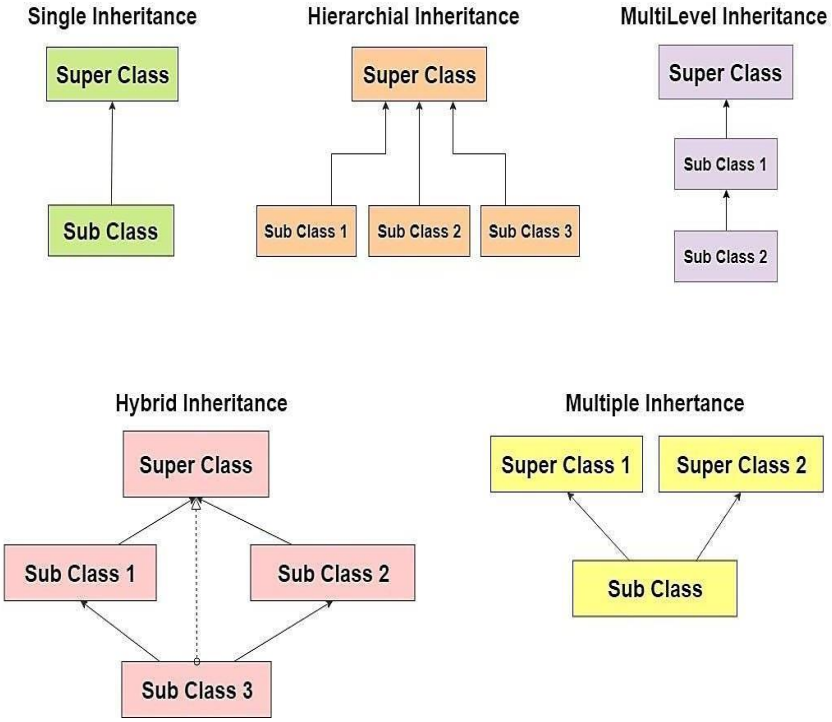
Output:

```
Animal eats food
Dog barks
```

Types of Inheritance:

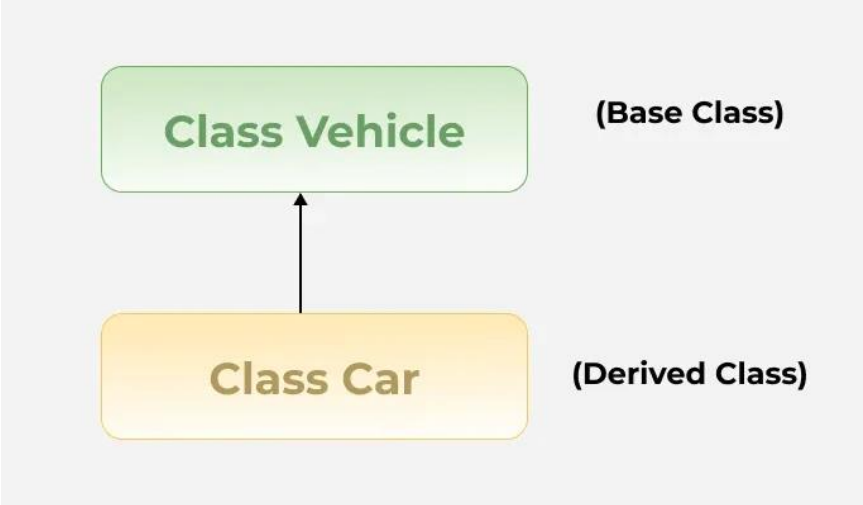
Below are the different types of inheritance which are supported by Java.

1. **Single Inheritance**
2. **Multilevel Inheritance**
3. **Hierarchical Inheritance**
4. **Multiple Inheritance**
5. **Hybrid Inheritance**



1. Single Inheritance:

- In single inheritance, a sub-class is derived from only one super class.
- It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



Single Inheritance Example:

```
class Animal
{
   void eat()
   {
      System.out.println("Animal eats food");
   }
}

class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog barks");
   }
}

public class InheritanceExample
{
   public static void main(String[] args)
   {
```

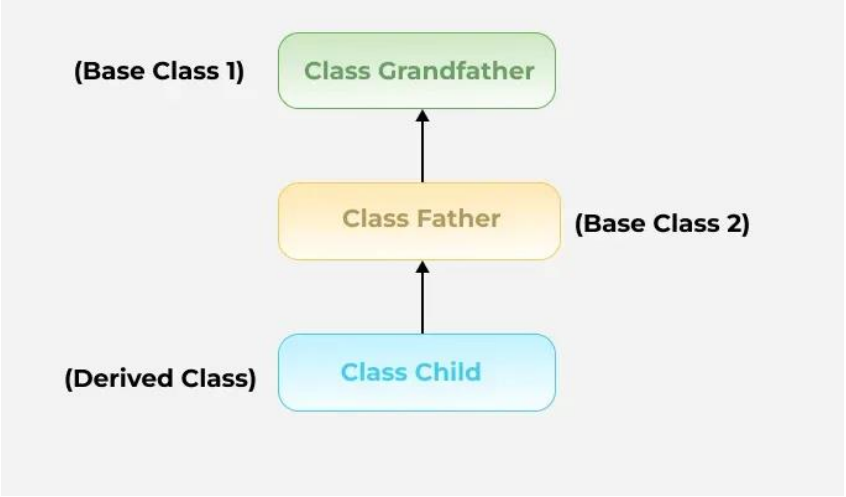
```
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output: **Animal eats**

Dog barks

2. Multilevel Inheritance:

- In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also acts as the base class for other classes.




```
//Multilevel Inheritance

class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
```

```
}

class Puppy extends Dog {
   void weep() {
      System.out.println("Puppy weeps");
   }
}


public class Test
{
   public static void main(String[] args) {
      Puppy p = new Puppy();
      p.eat();
      p.bark();
      p.weep();
   }
}
```

Output

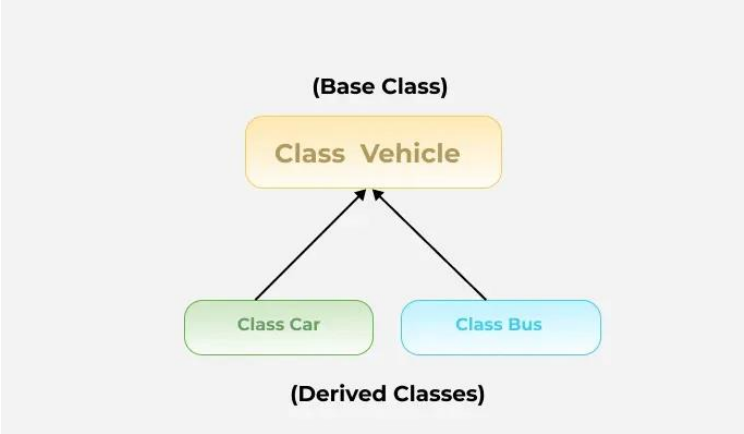
Animal eats

Dog barks

Puppy weeps

3. Hierarchical Inheritance

- In hierarchical inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.
- For example, cars and buses both are vehicle




```
//Hierarchical Inheritance

class Animal
{
   void eat()
   {
      System.out.println("Animal eats");
   }
}
class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog barks");
   }
}
class Cat extends Animal
{
   void meow()
   {
```

```
        System.out.println("Cat meows");
    }
}
public class Test
{
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();

        c.eat();
        c.meow();
    }
}
```

Output:

Animal eats
Dog barks
Animal eats
Cat meows

4. Multiple Inheritance (Through Interfaces)

- In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes.
- **Note:** that Java does not support multiple inheritances with classes.
- In Java, we can achieve multiple inheritances only through Interfaces.

Example on Inheritance:

Q1. b) Define class person with suitable data member and methods. And extend this class in Manager class. Display manager details.[5]

[Previous year paper Question: May 2025]

```
// Parent class
class Person
{
   int aadharID = 52341;
   String name = "Amit";
}

// Child class
class Manager extends Person
{
   int salary = 40000;
   int empid=101;

   void display()
   {
      System.out.println("Aadhar ID: " + aadharID);
      System.out.println("Name: " + name);
      System.out.println("Salary: " + salary);
      System.out.println("Employee Id: " + empid);
   }
}

// Main class
public class test
{
   public static void main(String[] args)
   {

      Manager m = new Manager();
      m.display();
```

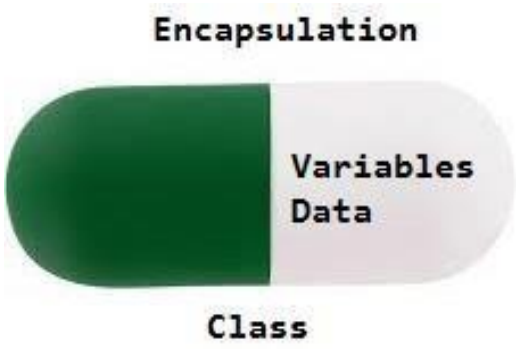
```
    }
}
```

Output:

```
Aadhar ID: 52341
Name: Amit
Salary: 40000
Employee Id: 101
```

➢ ENCAPSULATION:

- Encapsulation in Java is one of the fundamental principles of object-oriented programming (OOP).
- It refers to the practice of binding the data (fields) and methods (functions) that operate on the data into a single unit.
- Example: typically a class, and restricting direct access to some of the object's components.
- This is done to protect the integrity of the data and ensure controlled access and modification.



“Encapsulation means wrapping data (variables) and code (methods) together in a single unit (class) and restricting direct access to data using private access modifier.”

Example:

```
//Encapsulatiopn
class Student
{
   // private data members
   private int rollNo;
   private String name;

   // setter methods
   void setData(int r, String n)
   {
      rollNo = r;
      name = n;
   }

   // getter/display method
   void display() {
      System.out.println("Roll No: " + rollNo);
      System.out.println("Name: " + name);
   }
}

public class test
{
   public static void main(String[] args)
   {

      Student s = new Student();
      s.setData(101, "Rahul");
      s.display();
   }
}
```

Output:

Roll No: 101
Name: Rahul

➢ Advantages of Encapsulation

1. Data Security (Data Hiding)
Private variables cannot be accessed directly from outside the class.

2. Controlled Access
Data can be accessed or modified only through getter and setter methods.

3. Improves Maintainability
Code becomes easy to manage and update.

4. Better Code Organization
Data and methods are kept together in one class.

➢ Disadvantages of Encapsulation

1. More Code Writing
Need to create getter and setter methods.

2. Increases Code Length
Program becomes slightly longer.

1. 2.1 Abstract Class:

- An **abstract class** is a class that is declared using the abstract keyword and **cannot be instantiated (object cannot be created)**.

- It is used to achieve **abstraction** in Java.
- Abstract class is declared using abstract keyword
- Cannot create object of abstract class
- Abstract method must be implemented by child class
- Can have both abstract and normal methods

Why Abstract Class is Used?

- To hide implementation details
- To provide a common base class
- To force subclasses to implement certain methods
- To support partial abstraction
- **Partial abstraction** means:
  Some details are hidden, but some details are shown.
- An **abstract class** can have:
  ✓ Abstract methods → without body (hidden logic)
  ✓ Normal methods → with body (visible logic)

- So it does NOT hide everything — only some parts — that's why it is called **partial abstraction**.

➢ Simple Real-Life Idea

- Think of a **TV remote**
- You know buttons like ON/OFF (visible → normal method)
- But internal circuit working is hidden (abstract method)
- So only some things are hidden = partial abstraction

➢ Syntax of Abstract Class

```
abstract class ClassName
 {
   abstract void method1();   // abstract method
   void method2()
   {          // normal  method
      // method body
```

```
    }
}
```

Example 1: Simple Abstract Class

```
// Abstract class
abstract class Animal
{

    // Abstract method (no body)
    abstract void sound();

    // Normal method
    void eat()
    {
        System.out.println("Animal eats food");
    }
}

// Child class
class Dog extends Animal
{

    // Implement abstract method
    void sound()
    {
        System.out.println("Dog barks");
    }
}

// Main class
public class test
{
    public static void main(String[] args)
    {

        Dog d = new Dog();
        d.sound();   // Calling abstract method
        d.eat();     // Calling normal method
    }
}
```

Output:

Dog barks
Animal eats food

1.2.2 Interface

- An **interface** in Java is a blueprint of a class.
- It contains **only method declarations** (no method body), and the class that implements it must provide the implementation.
- It is used to achieve **100% abstraction**.

Key Points

- Declared using interface keyword
- Cannot create object of interface
- Methods are **public & abstract** by default
- Variables are public static final
- Class uses implements keyword
- Java does NOT support multiple inheritance with classes because of **ambiguity problem (diamond problem)**
- But Java supports multiple inheritance with interfaces

Easy Real-Life Example

- Imagine
- You ask **Mother** and **Father** how to cook rice
- Both give different instructions.
- Now you are confused — which one to follow?
- This confusion is the **diamond problem**.

Syntax

```
interface InterfaceName
{
   void methodName();   // abstract method
}

class ClassName implements InterfaceName
{

   public void methodName()
   {
      // method implementation
```

```
    }
}
```

Real-Life Example

- A student **extends** a person → because student is a type of person
- A student **implements** exam rules → because student follows rules

Example: Interface

```
// First interface
interface Father
{
   void showFather();
}
// Second interface
interface Mother
{
   void showMother();
}

// Class implementing both interfaces
class Child implements Father, Mother
{
   public void showFather()
   {
      System.out.println("This is Father method");
   }

  public void showMother()
   {
      System.out.println("This is Mother method");
   }
}
// Main class
public class Test
{
   public static void main(String[] args)
   {

      Child c = new Child();
      c.showFather();
```

```
        c.showMother();
    }
}
```

Output:

```
This is Father method
This is Mother method
```

➢ Abstract Class vs Interface :

Feature	Abstract Class	Interface
Definition	A class that can have both abstract and normal methods	A blueprint that contains only abstract methods
Abstraction Level	Partial abstraction	Full abstraction
Keyword Used	*abstract class*	*interface*
Methods	Can have abstract + concrete (normal) methods	Methods are abstract by default
Method Body	Allowed	Not allowed
Inheritance Keyword	*extends*	*implements*
Multiple Inheritance	Not supported	Supported
Access Modifiers	Can use private/protected/public	Methods are public by default
Speed	Faster (less abstraction)	Slightly slower
When to Use	When classes share common code	When only rules are needed
Relationship	Strong relationship (is-a)	Like contract (can-do)

1.4 Garbage Collector

Q.1 d) What is garbage collection? Explain with required method. [5]

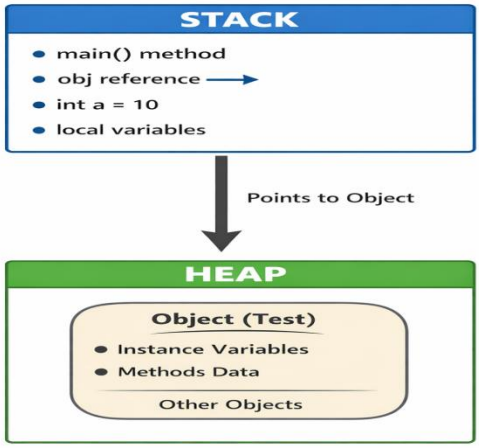
[Previous year question paper May 2025]

➢ What is Garbage Collector (GC)?

- Garbage Collector is a **memory management system** in Java that automatically removes unused objects from memory.
- It frees memory by deleting objects that are **no longer referenced** by any part of the program.
- This helps prevent **memory leaks** and improves performance.

Definition:

" Garbage Collection is a process that automatically destroys unused objects to free heap memory."



Real-Life Example

- Think of your computer desktop
- When files are not needed, you delete them to free space.
- Garbage Collector does the same with unused objects in memory.
-

Why Garbage Collection is Needed

- To free unused memory
- To avoid memory leaks
- To improve program performance
- To manage heap memory automatically
- No need for manual memory deletion

How Garbage Collector Works

- Java automatically checks objects in **Heap Memory**
- If an object has:

❌No reference
❌Not used anywhere

GC removes it and frees memory

Example of Garbage Collection

```
class Test {
   public static void main(String[] args) {

       Test t1 = new Test();
       Test t2 = new Test();

       t1 = null;  // eligible for GC
       t2 = null;  // eligible for GC

       System.gc(); // request GC
```

```
    }
}
```
Since objects have no reference → GC can remove them

Required Methods Related to Garbage Collection

1. System.gc()

- Requests JVM to run Garbage Collector
- It is NOT guaranteed to run immediately
- System.gc();

2. Runtime.getRuntime().gc()

- Another way to request GC

```
Runtime.getRuntime().gc();
```

3. finalize() Method

- Called by GC before destroying object
- Used to perform cleanup tasks
- **Example:**

```
class Test1
{

   protected void finalize()
   {
      System.out.println("Object destroyed");
   }

   public static void main(String[] args)
   {
      Test t = new Test();
      t = null;
      System.gc();
```

```
    }
}
```

1.5 Lambda expression

What is Lambda Expression?

- A **Lambda expression** is a short way to write anonymous functions .
- anonymous functions means function without name.
- It is used to provide implementation of **functional interfaces**.

- **Functional Interface**

Lambda works only with interface having **one abstract method.**

- Lambda expression is a concise way to represent a method without name using -> arrow symbol.

Syntax:

(argument list) -> {body of the expression}

- **Argument List:** Parameters for the lambda expression
- **Arrow Token (->):** Separates the parameter list and the body
- **Body:** Logic to be executed.

Example:

Q. 1. c) Write a function using lambda expression to calculate power of number.*(x^y)* *[5 marks]*

[Previous year question paper May 2025]

```
interface Power
{
   double calculate(int x, int y);
}

public class Test
{
   public static void main(String[] args)
   {

      // Lambda expression
      Power p = (x, y) -> Math.pow(x, y);

      // Calling function
      System.out.println("Power is: " + p.calculate(2, 3));
   }
}
```

Output:

Power is: 8.0

Explanation

1. Interface Power

Defines method calculate(x, y)

2. Lambda Expression

(x, y) -> Math.pow(x, y)

Takes two inputs and returns power

3. Math.pow()

Built-in method to calculate power

4. Calling method

p.calculate(2,3) → $2^3 = 8$

➢ Previous Year Questions:

1. Define a class calculation to implement method overloading for addition of two integers and two double variables. **[5]**

2. Define class person with suitable data member and methods. And extend this class in Manager class. Display manager details. **[5]**

3. Write a function using lambda expression to calculate power of number.(x^y) **[5]**

4. What is garbage collection? Explain with required method. **[5]**